

Compte Rendu — TP2 Introduction Flask

Étudiant : Yanni
Date : 11 mai 2026
Objectif : Serveur Backend Flask, MySQL et sécurisation JWT

1. Installation de l'environnement

Python et Flask

```
sudo apt install python3 python3-pip  
pip install Flask
```

Configuration des variables d'environnement :

```
export FLASK_APP=app.py  
export FLASK_ENV=development
```

Lancement du serveur :

```
python3 -m flask run
```

2. Création du serveur et des premières routes

Création du dossier projet :

```
mkdir tp2_flask  
cd tp2_flask  
nano app.py
```

Route /

```
@app.route('/')  
def index():  
    return 'Bienvenue sur mon serveur Flask !'
```

Route /about

```
@app.route('/about')  
def about():  
    return 'Bonjour je suis Yanni'
```

Route /hello/

```
@app.route('/hello/<prenom>')  
def hello(prenom):  
    return f'Bonjour a toi {prenom} mon ami(e) !'
```

Résultats obtenus dans le navigateur :

- `http://127.0.0.1:5000/` → Bienvenue sur mon serveur Flask !
- `http://127.0.0.1:5000/about` → Bonjour je suis Yanni
- `http://127.0.0.1:5000/hello/Yanni` → Bonjour a toi Yanni mon ami(e) !

3. Installation et configuration de MySQL

```
sudo apt install mysql-server
sudo mysql_secure_installation
```

Configuration du mot de passe root :

```
sudo mysql

ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'password';
FLUSH PRIVILEGES;
exit;
```

Création de la base de données et de la table :

```
CREATE DATABASE db_tp2;
USE db_tp2;
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pseudo VARCHAR(25),
    password VARCHAR(255)
);
SHOW TABLES;
SELECT * FROM users;
```

4. Installation du package MySQL pour Python

```
pip install mysql-connector-python
```

5. Routes avec base de données

Connexion à la base de données

```
import mysql.connector as MC

try:
    db = MC.connect(
        host="localhost",
        user="root",
        password="password",
        database="db_tp2"
    )
    print("Connexion a la DB reussie !")
except MC.Error as e:
    print(f"Erreur de connexion : {e}")
```

Route /users — Afficher tous les utilisateurs

```
@app.route('/users')
def users():
    cursor = db.cursor()
    cursor.execute("SELECT pseudo FROM users")
    result = cursor.fetchall()
    return jsonify(result)
```

Route /user/ — Afficher un utilisateur

```
@app.route('/user/<pseudo>')
def user(pseudo):
    cursor = db.cursor()
    cursor.execute("SELECT id, pseudo FROM users WHERE pseudo = %s", (pseudo,))
    result = cursor.fetchone()
    if result is None:
        return jsonify({"message": "not found"})
    return jsonify({"id": result[0], "pseudo": result[1]})
```

Route /create — Créer un utilisateur (POST)

```
@app.route('/create', methods=['POST'])
def create():
    pseudo = request.form.get('pseudo')
    password = request.form.get('password')
    if not pseudo or not password:
        return jsonify({"message": "pseudo et password obligatoires"})
    cursor = db.cursor()
    cursor.execute("INSERT INTO users (pseudo, password) VALUES (%s, %s)", (pseudo, password))
    db.commit()
    return jsonify({"message": "utilisateur cree !"})
```

Tests avec curl :

```
# Creer un utilisateur
curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/create \
-d "pseudo=user1&password=1234"
# -> {"message": "utilisateur cree !"}

# Afficher tous les utilisateurs
curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/users
# -> [{"user1"}]

# Afficher un utilisateur existant
curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/user/user1
# -> {"id": 1, "pseudo": "user1"}

# Afficher un utilisateur inexistant
curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/user/maxime
# -> {"message": "not found"}
```

6. Sécurisation de l'API avec JWT

```
pip install PyJWT
```

```
import jwt
import datetime
```

```
SECRET_KEY = "cle_secrete"
```

Route /login — Génération du token

La route reçoit le pseudo et le mot de passe, vérifie les identifiants en base de données, puis génère un token JWT valable 30 minutes contenant le pseudo de l'utilisateur.

```
@app.route('/login', methods=['POST'])
def login():
    pseudo = request.form.get('pseudo')
    password = request.form.get('password')
```

```

if not pseudo or not password:
    return jsonify({"message": "pseudo et password obligatoires"})
cursor = db.cursor()
cursor.execute("SELECT * FROM users WHERE pseudo = %s AND password = %s", (pseudo, password))
result = cursor.fetchone()
if result is None:
    return jsonify({"message": "identifiants incorrects"})
token = jwt.encode({
    "pseudo": pseudo,
    "exp": datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
}, SECRET_KEY, algorithm="HS256")
return jsonify({"token": token})

```

Récupération du token via /login :

```

curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/login \
-d "pseudo=user1&password=1234"
# -> {"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..." }

```

Le token retourné est ensuite copié et passé dans le header Authorization de toutes les requêtes sécurisées.

Route /users — Sécurisée par token

```

@app.route('/users')
def users():
    token = request.headers.get('Authorization')
    if not token:
        return jsonify({"message": "Token manquant"}), 401
    try:
        jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    except:
        return jsonify({"message": "Token invalide"}), 401
    cursor = db.cursor()
    cursor.execute("SELECT pseudo FROM users")
    result = cursor.fetchall()
    return jsonify(result)

```

1. Sans token -> refus

```

curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/users
# -> {"message": "Token manquant"}

```

2. Récupération du token via /login

```

curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/login \
-d "pseudo=user1&password=1234"
# -> {"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..." }

```

3. Utilisation du token récupéré depuis /login

```

curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/users \
-H "Authorization: eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
# -> [{"user1"}]

```

Route /me — Données du token

```

@app.route('/me')
def me():
    token = request.headers.get('Authorization')
    if not token:
        return jsonify({"message": "Token manquant"}), 401
    try:

```

```

        data = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return jsonify({"message": f"Tu es connecte(e) sous le pseudo de : {data['pseudo']}"})
    except:
        return jsonify({"message": "Token invalide"}), 401

# 1. Recuperation du token via /login
curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/login \
-d "pseudo=user1&password=1234"
# -> {"token": "eyJ0eXAiOiJKV1Qi..." }

# 2. Utilisation du token sur /me
curl --no-proxy 127.0.0.1 http://127.0.0.1:5000/me \
-H "Authorization: eyJ0eXAiOiJKV1Qi..."
# -> {"message": "Tu es connecte(e) sous le pseudo de : user1"}

```

7. Partie Bonus — Modification du mot de passe

Critères du nouveau mot de passe :

- Longueur min : ≥ 6 caractères
- Longueur max : ≤ 10 caractères
- Au moins 1 majuscule
- Au moins 1 minuscule
- Au moins 1 chiffre
- Différent du mot de passe actuel

```

@app.route('/update_password', methods=['POST'])
def update_password():
    token = request.headers.get('Authorization')
    if not token:
        return jsonify({"message": "Token manquant"}), 401
    try:
        data = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    except:
        return jsonify({"message": "Token invalide"}), 401
    pseudo = data['pseudo']
    new_password = request.form.get('password')
    if not new_password:
        return jsonify({"message": "password obligatoire"})
    if len(new_password) < 6:
        return jsonify({"message": "Mot de passe trop court (min 6)"}), 400
    if len(new_password) > 10:
        return jsonify({"message": "Mot de passe trop long (max 10)"}), 400
    if not any(c.isupper() for c in new_password):
        return jsonify({"message": "Doit contenir au moins 1 majuscule"}), 400
    if not any(c.islower() for c in new_password):
        return jsonify({"message": "Doit contenir au moins 1 minuscule"}), 400
    if not any(c.isdigit() for c in new_password):
        return jsonify({"message": "Doit contenir au moins 1 chiffre"}), 400
    cursor = db.cursor()
    cursor.execute("SELECT password FROM users WHERE pseudo = %s", (pseudo,))
    result = cursor.fetchone()
    if result[0] == new_password:
        return jsonify({"message": "Le mot de passe doit etre different de l'actuel"}), 400

```

```

        cursor.execute("UPDATE users SET password = %s WHERE pseudo = %s", (new_password, pseudo))
        db.commit()
        return jsonify({"message": "Mot de passe modifie !"})

# 1. Recuperation du token via /login
curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/login \
-d "pseudo=user1&password=1234"
# -> {"token": "eyJ0eXAiOiJKV1Qi..." }

# 2. Modification du mot de passe avec le token recupere
curl --no-proxy 127.0.0.1 -X POST http://127.0.0.1:5000/update_password \
-H "Authorization: eyJ0eXAiOiJKV1Qi..." \
-d "password=NewPass1"
# -> {"message": "Mot de passe modifie !"}

```

8. Récapitulatif des routes

Route	Méthode	Token	Description
/	GET	Non	Page d'accueil
/about	GET	Non	Affiche le prénom
/hello/<prenom>	GET	Non	Message personnalisé
/users	GET	Oui	Liste des pseudos
/user/<pseudo>	GET	Non	Détails d'un utilisateur
/create	POST	Non	Créer un utilisateur
/login	POST	Non	Génère un token JWT (30 min)
/me	GET	Oui	Affiche le pseudo connecté
/update_password	POST	Oui	Modifie le mot de passe